Seminar: Data Literacy

Part 1: Designing Experiments to Collect Data

Data is important, but the more important is how we can collect desired data effectively. For example, if we designed a new product and had a new design, how should we evaluate its usefulness and effectiveness? We need to design an **experiment** (Exploratory or Hypothesis driven)!

There are a few main elements of an **experiment**:

- Participant
- Apparatus
- Experiment Procedure
- Design
- Analysis

Participant

Who are they? Where do they come from?

Question: how many participants do we need for the test?

Increasing the number of participants enhances your study's statistical power, or its capacity to identify differences among the variables being compared. However, the required sample size depends on how large those differences are. If the variables in question differ greatly, fewer participants are needed; if they are relatively similar, a larger number of participants may be necessary.

Would identifying a difference with a sample of 60 participants be sufficient to assure that this statistically significant finding also has practical importance? It is essential to remember that statistical significance and practical significance, which stems from a meaningful real-world difference, are two distinct considerations. Furthermore, every experiment must account for at least four major factors.

Sampling

Generally speaking, there are two types of sampling methods: *probability sampling* and *non-probability sampling*

probability sampling In probability sampling, every unit in the target population has a known and non-zero probability of being selected.

non-probability sampling In non-probability sampling, elements are selected based on factors such as convenience, judgment, or other non-random criteria. Examples include purposive sampling, convenience sampling, and snowball sampling.

- Purposive sampling: Researchers select units based on specific criteria or their judgment regarding who can provide the most relevant information.
- Convenience sampling: Participants are selected based on ease of access or availability (e.g., surveying people passing by on a street).
- Snowball sampling: Existing study participants recruit or nominate future participants from among their acquaintances, often used when the population is hard to reach or is a specialized group.

Apparatus

- Equipment
- Space
- Other resources to run the study

Procedure

- What do participants do during the experiment?

Design

- Do they perform tasks, what kinds, and how many?
- How long do they take?
- Do they get to practice something beforehand or be exposed for a while beforehand before we start measuring what data that we want to count?

Fundamental Statistical Method

Test of Proportion

Used with categorical data, typically binary (yes/bim success/failure). It compares proportions or percentages between groups or against a reference value A test of proportion is where you're looking at a proportion of responses. If you find yourself counting subjects themselves, for example, each subject gives their preference, then you may be doing a test of proportions. It's called **one sample**, because we have a **sample** of their preference. So on one variable preference, we have proportions whether they liked website A or B. So that's a one sample test. In a little bit, we'll also see a two sample test of proportions.

Assumptions

- Independent observations
- Large enough sample size for normal approximation

Analyses of Variance (ANOVA)

Used with continuous/interval data (measurements like height, weight, scores). It compares means between two or more groups. **Assumptions**

- Normal distribution within groups
- Homogeneity of variance
- Independent observations (except in repeated measures designs)

Case Study: Applying Statistical Inference on Product Preference Data

Objective

In this example, we explore how to use **statistical inference** to determine whether **product preference** (Product A vs. Product B) is **associated with participant sex** (Male vs. Female). This process illustrates the key steps in applying statistical inference to real-world categorical data.

---

1. **Understanding the Data**

We have a dataset (`prefsABsex.csv`) where:

- Each row represents a participant.
- `Pref` indicates their preferred product: **A** or **B**.
- `Sex` indicates the participant's sex: **M** (Male) or **F** (Female).

1.1 **Setup and Data Import**

```
# Install required libraries
!pip install pandas numpy scipy matplotlib seaborn kagglehub

# Import libraries
import os
import sys
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df = pd.read_csv('prefsABsex.csv')

# Display the first few rows of the dataset
print("First few rows of the dataset:")
print(df.head())
```

```
First few rows of the dataset:
   Subject Pref Sex
0        1    B   F
1        2    B   F
2        3    B   F
```

```
3        4    B   M
4        5    B   F

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60 entries, 0 to 59
Data columns (total 3 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Subject  60 non-null     int64
 1   Pref     60 non-null     object
 2   Sex      60 non-null     object
dtypes: int64(1), object(2)
memory usage: 1.5+ KB
```

1.2 **Data Exploration**

```
# Basic information about the dataset
print("\nDataset Info:")
df.info()

# Summary statistics
print("\nSummary Statistics:")
df.describe(include='all')


Summary Statistics:

          Subject Pref  Sex
count   60.000000   60   60
unique        NaN    2    2
top           NaN    B    F
freq          NaN   46   31
mean    30.500000  NaN  NaN
std     17.464249  NaN  NaN
min      1.000000  NaN  NaN
25%     15.750000  NaN  NaN
50%     30.500000  NaN  NaN
75%     45.250000  NaN  NaN
max     60.000000  NaN  NaN

# Check for missing values
print("\nMissing Values:")
df.isnull().sum()


Missing Values:

Subject    0
Pref       0
```

```
Sex         0
dtype: int64
```

Let's check the distribution of our categorical variables:

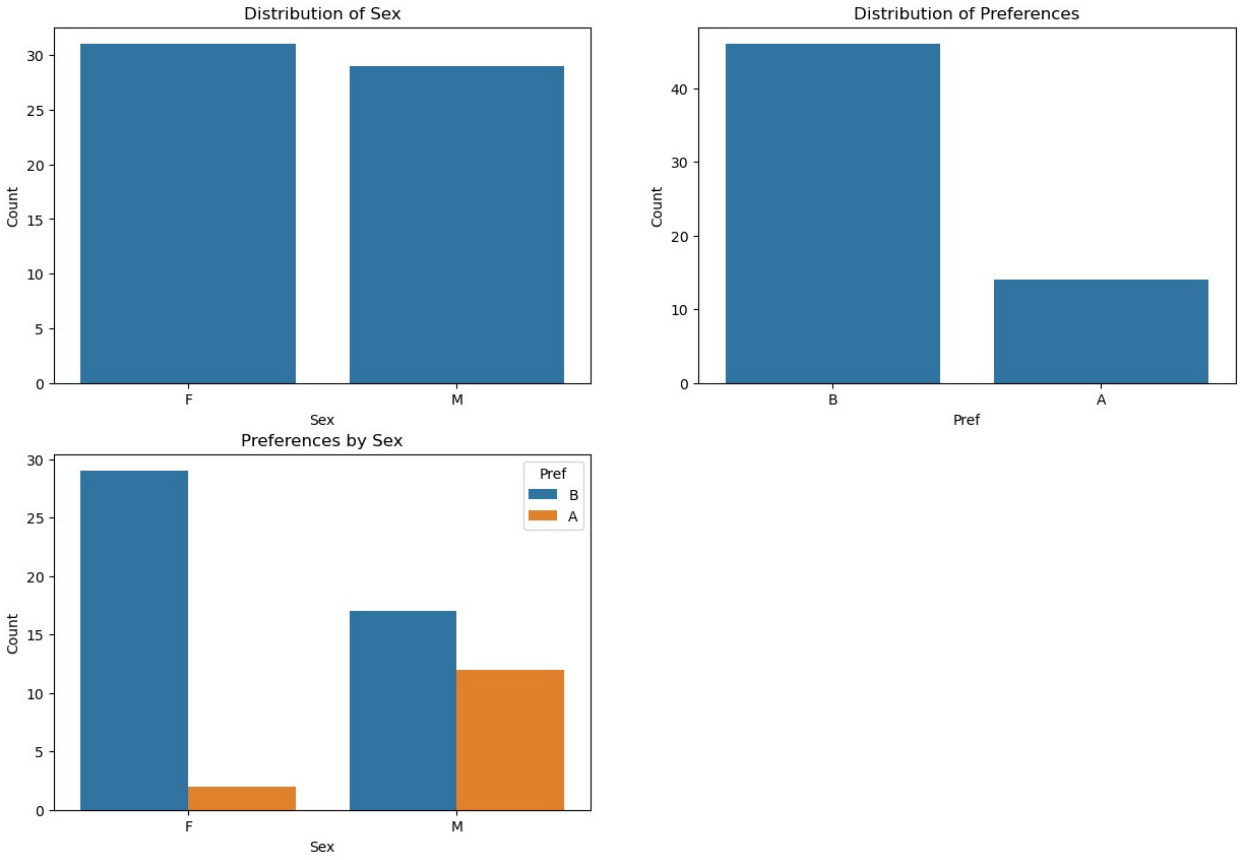
```python
# Distribution of Sex
print("\nDistribution of Sex:")
sex_counts = df['Sex'].value_counts()
print(sex_counts)

# Distribution of Preferences
print("\nDistribution of Preferences:")
pref_counts = df['Pref'].value_counts()
print(pref_counts)

# Cross-tabulation of Sex and Preferences
print("\nCross-tabulation of Sex and Preferences:")
pd.crosstab(df['Sex'], df['Pref'])
```

```
Distribution of Sex:
Sex
F    31
M    29
Name: count, dtype: int64

Distribution of Preferences:
Pref
B    46
A    14
Name: count, dtype: int64

Cross-tabulation of Sex and Preferences:

Pref   A   B
Sex
F      2  29
M     12  17
```

1.3. **Data Visualization**

```python
plt.figure(figsize=(15, 10))

# Plot 1: Bar chart of Sex distribution
plt.subplot(2, 2, 1)
sns.countplot(x='Sex', data=df)
plt.title('Distribution of Sex')
plt.ylabel('Count')
```

```
# Plot 2: Bar chart of Preference distribution
plt.subplot(2, 2, 2)
sns.countplot(x='Pref', data=df)
plt.title('Distribution of Preferences')
plt.ylabel('Count')

# Plot 3: Bar chart of Preferences by Sex
plt.subplot(2, 2, 3)
sns.countplot(x='Sex', hue='Pref', data=df)
plt.title('Preferences by Sex')
plt.ylabel('Count')


plt.show()
```



Motivation

We are interested in answering the following research question:

Is there a significant relationship between sex and product preference?

2. Formulating Hypotheses

We apply a **Chi-Square Test of Independence**, which is suitable when both variables are categorical.

- **Null Hypothesis (H_0)**: Sex and product preference are **independent** (i.e., there is no association).
- **Alternative Hypothesis (H_1)**: Sex and product preference are **not independent** (i.e., there is an association).

---

3. Creating a Contingency Table

We begin by creating a **contingency table**, which summarizes the frequency of each combination of preference and sex:

Preference	Female (F)	Male (M)
A	2	12
B	29	17

This table shows how many males and females preferred each product.

```
# Create a contingency table
contingency_table = pd.crosstab(df['Sex'], df['Pref'])
print("Contingency Table:")
print(contingency_table)
```

```
Contingency Table:
Pref   A   B
Sex
F      2  29
M     12  17
```

---

4. Applying the Chi-Square Test

We use the **Chi-Square Test of Independence** to determine if the observed frequencies differ significantly from what we would expect if there were no association.

Observed Frequencies:

From the data above.

Expected Frequencies:

Calculated by the formula:

[E_{ij} = \frac{(\text{Row Total}) \times (\text{Column Total})}{\text{Grand Total}}]

For example:

- Expected females preferring Product A = (Total A) × (Total F) / N = (14 × 31) / 60 ≈ 7.23

```
from scipy import stats

chi2, p, dof, expected = stats.chi2_contingency(contingency_table)

print("\nChi-Square Test Results:")
print(f"Chi-Square Value: {chi2:.4f}")
print(f"p-value: {p:.4f}")
print(f"Degrees of Freedom: {dof}")
print("\nExpected Frequencies:")
print(pd.DataFrame(expected, index=contingency_table.index,
columns=contingency_table.columns))


Chi-Square Test Results:
Chi-Square Value: 8.3588
p-value: 0.0038
Degrees of Freedom: 1

Expected Frequencies:
Pref         A          B
Sex
F     7.233333  23.766667
M     6.766667  22.233333
```

5. Test Output

- **Chi-Square Statistic**: 8.36
- **Degrees of Freedom (df)**: 1
- **P-Value**: 0.0038
- **Significance Level (α)**: 0.05

```
# Interpret the result
alpha = 0.05
print("\nInterpretation:")
if p < alpha:
    print(f"With p-value = {p:.4f} < {alpha}, we reject the null
hypothesis.")
    print("There is a significant association between Sex and
Preferences.")
else:
```

```
    print(f"With p-value = {p:.4f} > {alpha}, we fail to reject the
null hypothesis.")
    print("There is no significant association between Sex and
Preferences.")


Interpretation:
With p-value = 0.0038 < 0.05, we reject the null hypothesis.
There is a significant association between Sex and Preferences.
```

6. Decision and Interpretation

Since the **p-value (0.0038) < α (0.05)**, we **reject the null hypothesis**.

> **Conclusion**: There is a statistically significant association between **sex** and **product preference**. Preference is not evenly distributed across sexes; for instance, more males prefer Product A, while more females prefer Product B.

Conclusion

- **Chi-Square Test** is a powerful tool for testing relationships between categorical variables.
- Always check assumptions: data must be counts, and expected frequencies should generally be ≥ 5.
- A small p-value means we have evidence to suggest a real association in the population.

Case Study: Mobile Keyboard and Posture Effects Analysis

1. Setup and Data Import

```
import statsmodels.api as sm
from statsmodels.formula.api import ols
import statsmodels.stats.multicomp as mc

# Read the CSV file
df = pd.read_csv('mbltxt.csv')

# Display the first few rows
print("First 5 rows of the dataset:")
df.head()

First 5 rows of the dataset:
```

```
   Subject Keyboard Posture  Posture_Order      WPM  Error_Rate
0        1   iPhone     Sit              1  20.2145      0.0220
1        1   iPhone   Stand              2  23.7485      0.0300
2        1   iPhone    Walk              3  20.7960      0.0415
3        2   iPhone     Sit              1  20.8805      0.0220
4        2   iPhone   Stand              3  23.2595      0.0255
```

2. Data Exploration

```
# Basic information about the dataset
print("\nDataset Info:")
df.info()

# Summary statistics
print("\nSummary Statistics:")
df.describe()


Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72 entries, 0 to 71
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Subject        72 non-null     int64
 1   Keyboard       72 non-null     object
 2   Posture        72 non-null     object
 3   Posture_Order  72 non-null     int64
 4   WPM            72 non-null     float64
 5   Error_Rate     72 non-null     float64
dtypes: float64(2), int64(2), object(2)
memory usage: 3.5+ KB

Summary Statistics:

         Subject  Posture_Order        WPM  Error_Rate
count  72.000000      72.000000  72.000000   72.000000
mean   12.500000       2.000000  20.212507    0.033812
std     6.970764       0.822226   4.136840    0.015414
min     1.000000       1.000000   9.453500    0.015000
25%     6.750000       1.000000  19.090875    0.022000
50%    12.500000       2.000000  21.032500    0.030500
75%    18.250000       3.000000  23.476125    0.040000
max    24.000000       3.000000  25.380000    0.069500

# Check for missing values
print("\nMissing Values:")
df.isnull().sum()
```

```
Missing Values:

Subject          0
Keyboard         0
Posture          0
Posture_Order    0
WPM              0
Error_Rate       0
dtype: int64
```

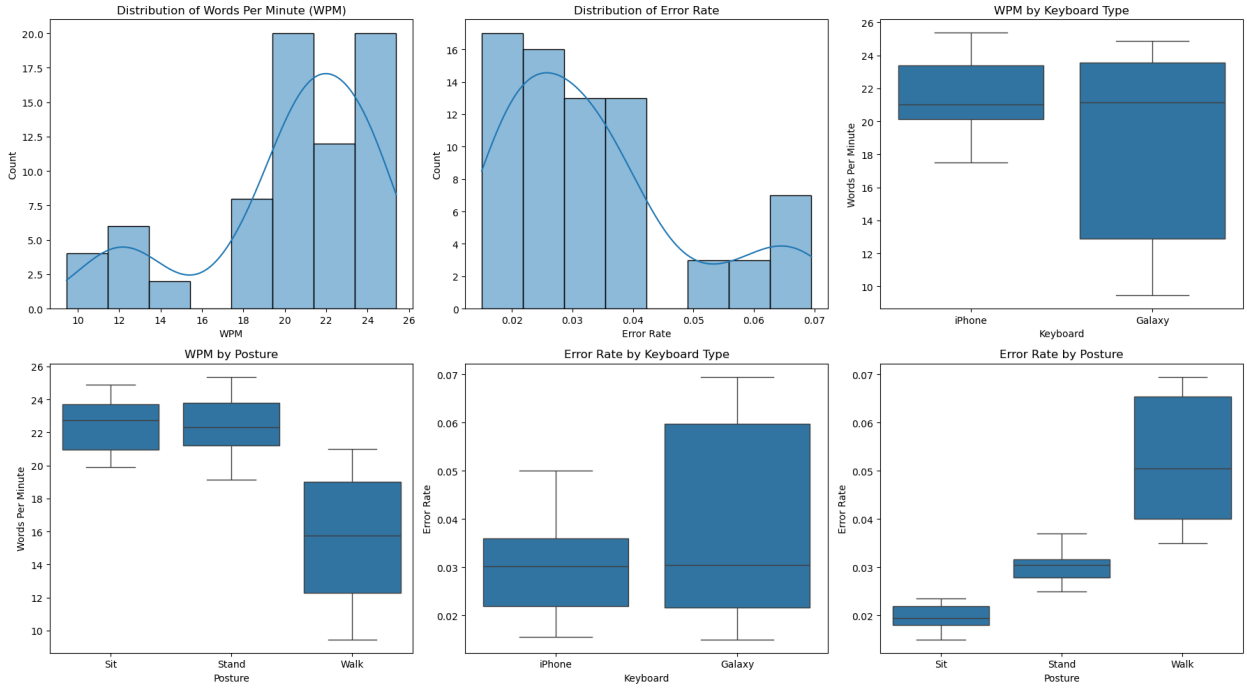
```
# Distribution of Keyboard
print("\nDistribution of Keyboard types:")
print(df['Keyboard'].value_counts())

# Distribution of Posture
print("\nDistribution of Posture types:")
print(df['Posture'].value_counts())

# Cross-tabulation of Keyboard and Posture
print("\nCross-tabulation of Keyboard and Posture:")
pd.crosstab(df['Keyboard'], df['Posture'])
```

```
Distribution of Keyboard types:
Keyboard
iPhone    36
Galaxy    36
Name: count, dtype: int64

Distribution of Posture types:
Posture
Sit      24
Stand    24
Walk     24
Name: count, dtype: int64

Cross-tabulation of Keyboard and Posture:

Posture   Sit  Stand  Walk
Keyboard
Galaxy     12     12    12
iPhone     12     12    12
```

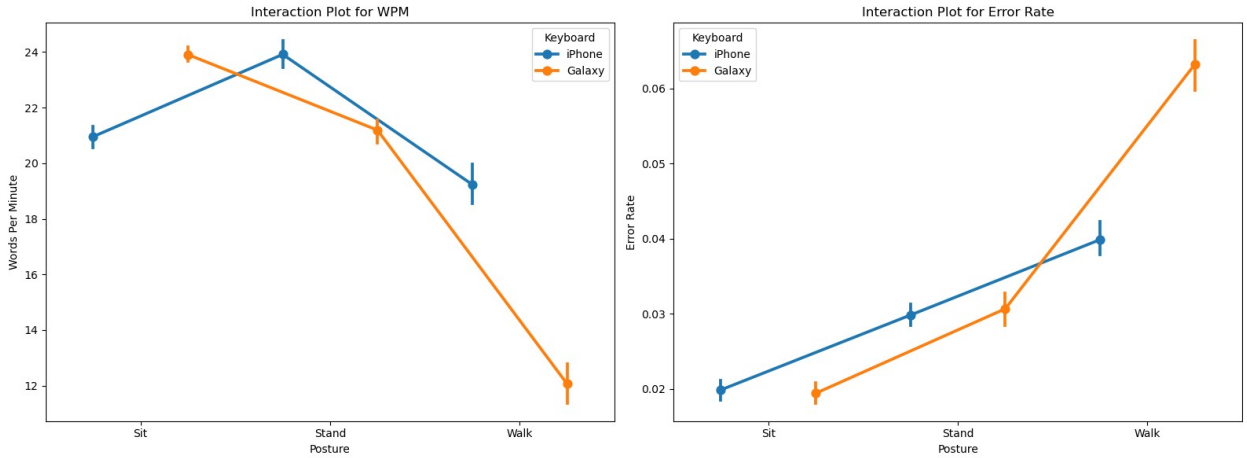
3. Data Visualization

```
# Create a figure with multiple subplots for easy comparison
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Plot 1: Distribution of WPM overall
sns.histplot(df['WPM'], kde=True, ax=axes[0, 0])
```

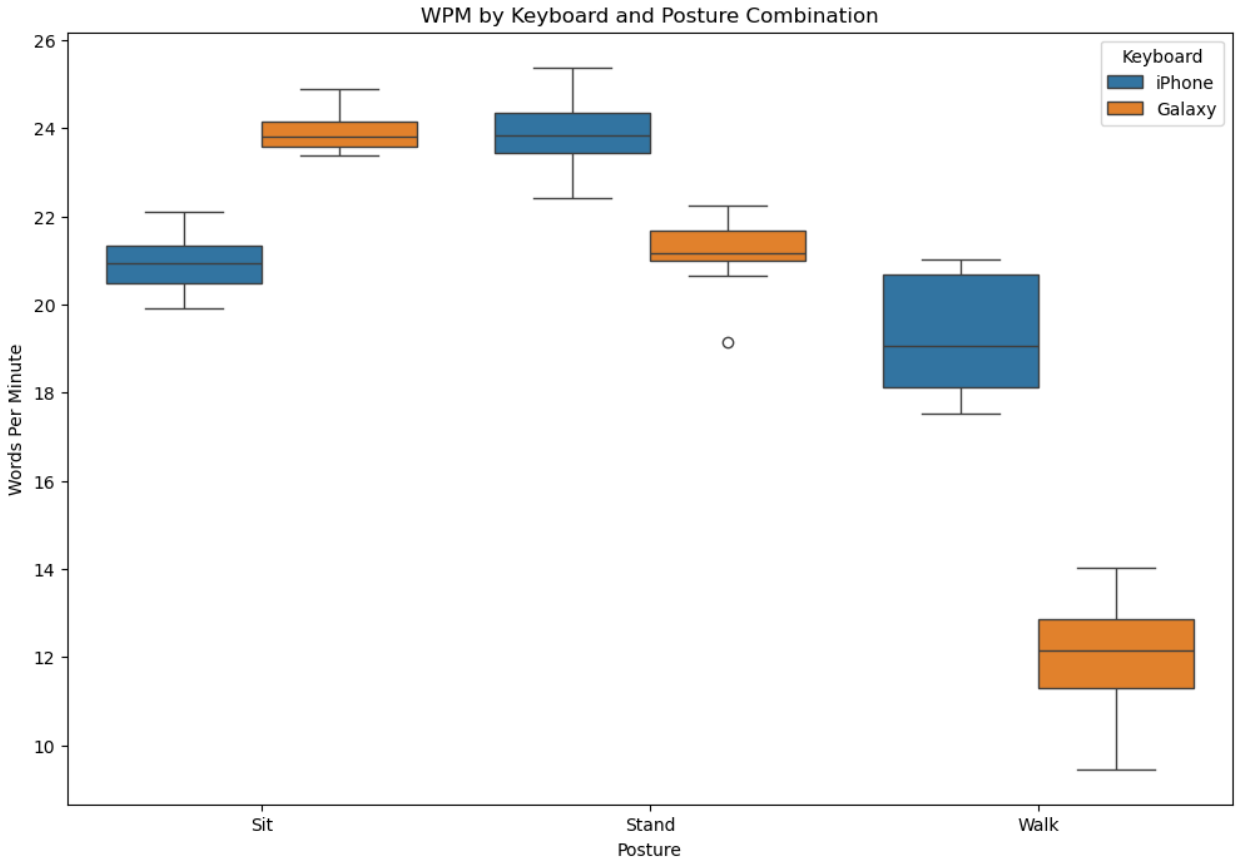
```
axes[0, 0].set_title('Distribution of Words Per Minute (WPM)')
axes[0, 0].set_xlabel('WPM')

# Plot 2: Distribution of Error Rate overall
sns.histplot(df['Error_Rate'], kde=True, ax=axes[0, 1])
axes[0, 1].set_title('Distribution of Error Rate')
axes[0, 1].set_xlabel('Error Rate')

# Plot 3: WPM by Keyboard
sns.boxplot(x='Keyboard', y='WPM', data=df, ax=axes[0, 2])
axes[0, 2].set_title('WPM by Keyboard Type')
axes[0, 2].set_xlabel('Keyboard')
axes[0, 2].set_ylabel('Words Per Minute')

# Plot 4: WPM by Posture
sns.boxplot(x='Posture', y='WPM', data=df, ax=axes[1, 0])
axes[1, 0].set_title('WPM by Posture')
axes[1, 0].set_xlabel('Posture')
axes[1, 0].set_ylabel('Words Per Minute')

# Plot 5: Error Rate by Keyboard
sns.boxplot(x='Keyboard', y='Error_Rate', data=df, ax=axes[1, 1])
axes[1, 1].set_title('Error Rate by Keyboard Type')
axes[1, 1].set_xlabel('Keyboard')
axes[1, 1].set_ylabel('Error Rate')

# Plot 6: Error Rate by Posture
sns.boxplot(x='Posture', y='Error_Rate', data=df, ax=axes[1, 2])
axes[1, 2].set_title('Error Rate by Posture')
axes[1, 2].set_xlabel('Posture')
axes[1, 2].set_ylabel('Error Rate')

plt.tight_layout()
plt.show()
```




```
# Create a new figure for interaction plots
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Interaction plot for WPM
sns.pointplot(x='Posture', y='WPM', hue='Keyboard', data=df,
dodge=0.5, ax=axes[0])
axes[0].set_title('Interaction Plot for WPM')
axes[0].set_xlabel('Posture')
axes[0].set_ylabel('Words Per Minute')

# Plot 2: Interaction plot for Error Rate
sns.pointplot(x='Posture', y='Error_Rate', hue='Keyboard', data=df,
dodge=0.5, ax=axes[1])
axes[1].set_title('Interaction Plot for Error Rate')
axes[1].set_xlabel('Posture')
axes[1].set_ylabel('Error Rate')

plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(12, 8))
sns.boxplot(x='Posture', y='WPM', hue='Keyboard', data=df)
plt.title('WPM by Keyboard and Posture Combination')
plt.xlabel('Posture')
plt.ylabel('Words Per Minute')
plt.legend(title='Keyboard')
plt.show()
```



plt.figure(figsize=(12, 8)) sns.boxplot(x='Posture', y='Error_Rate', hue='Keyboard', data=df) plt.title('Error Rate by Keyboard and Posture Combination') plt.xlabel('Posture') plt.ylabel('Error Rate') plt.legend(title='Keyboard') plt.show()

4. Statistical Analysis

4.1 Two-way ANOVA for WPM

```
# Create a model formula for WPM
model_wpm = ols('WPM ~ C(Keyboard) + C(Posture) +
C(Keyboard):C(Posture)', data=df).fit()

# Generate ANOVA table
anova_table_wpm = sm.stats.anova_lm(model_wpm, typ=2)

print("Two-way ANOVA Results for WPM:")
print(anova_table_wpm)

# Interpret the results
alpha = 0.05
print("\nInterpretation of WPM ANOVA:")
for effect, p_value in anova_table_wpm['PR(>F)'].items():
    if effect != 'Residual':
        if p_value < alpha:
            print(f"- {effect}: Significant effect (p =
{p_value:.4f})")
        else:
            print(f"- {effect}: No significant effect (p =
{p_value:.4f})")
```

```
Two-way ANOVA Results for WPM:
                            sum_sq    df           F        PR(>F)
C(Keyboard)              96.346864   1.0  105.513961  2.541998e-15
C(Posture)              749.628503   2.0  410.476631  5.810662e-38
C(Keyboard):C(Posture)  308.813269   2.0  169.097933  1.065168e-26
Residual                 60.265893  66.0         NaN           NaN

Interpretation of WPM ANOVA:
- C(Keyboard): Significant effect (p = 0.0000)
- C(Posture): Significant effect (p = 0.0000)
- C(Keyboard):C(Posture): Significant effect (p = 0.0000)
```

4.2 Two-way ANOVA for Error Rate

Can you complete this analysis by yourself?